

RelationshipOS

Full-Stack Customer Retention & Churn Prediction System

A production-ready full-stack MERN web application built as an entry for the Epsilon TeXpedition Hackathon. RelationshipOS leverages key client health telemetry to calculate attrition risk via customized heuristics, generating actionable AI marketing and retention strategies with Gemini integrations and robust offline fail-safes.

SUBMISSION PROFILE & SPECIFICATIONS

Hackathon : [Epsilon TeXpedition 2026](#)
Primary Stack : [MERN \(MongoDB Atlas, Express.js, React, Node.js\)](#)
AI Provider : [Google Gemini API \(gemini-2.0-flash-lite\)](#)
Local Workspaces : [d:/RelationshipOS](#)
Live Frontend URL : <https://relationship-os-psi.vercel.app/>
Live Backend URL : <https://relationship-os-backend.vercel.app/>
GitHub Repository : <https://github.com/RishaubKumar/RelationshipOS.git>

Designed for the Epsilon TeXpedition Evaluation Committee

1. Executive Summary

In modern subscription-based business models, customer churn (attrition) represents a major threat to profitability and sustainable growth. Acquiring a new customer is up to five times more expensive than retaining an existing one. RelationshipOS addresses this business-critical problem by offering a centralized dashboard designed for customer success managers and marketing departments to analyze customer activity, calculate churn risk in real-time, and generate direct action plans to retain at-risk users.

Built specifically within the bounds of a hackathon format, the application prioritizes speed, absolute runtime reliability, and clean UX design. RelationshipOS achieves this by employing a hybrid architecture: using deterministic heuristic algorithms for immediate, error-free churn analysis, and integrating Gemini's Generative AI models to dynamically draft custom marketing campaigns and incentives. Additionally, to guarantee continuous operations during live judges' evaluations, a local heuristic advisor engine acts as an instantaneous fallback if the API rate limit is exceeded.

2. System Architecture

RelationshipOS is engineered as a full-stack Javascript application using the MERN stack. Both the frontend and backend are fully decoupled and deployed independently on the Vercel Serverless cloud platform.

Architectural Flow

1. The React SPA (Frontend) authenticates users via JWT. The application maintains state locally and sends authenticated API calls using Axios.
2. The Express API (Backend) processes CRUD requests, triggers mathematical churn scoring, and manages data streams.
3. MongoDB Atlas (Database Cluster) serves as the persistent cloud store, whitelisting requests from all networks to accommodate dynamic serverless function IPs.
4. Google Gemini API is called asynchronously for AI recommendations, falling back to a rule-based algorithm in the event of an API quota cap (HTTP 429).

Vercel Serverless Integration & Cold Start Handling

Deploying Node.js applications as serverless functions on Vercel introduces cold-start challenges, where database connections must initialize within milliseconds. In standard apps, Mongoose buffers query executions globally. If the database connection is not ready, queries hang and eventually time out, causing 500 errors.

RelationshipOS resolves this by implementing an asynchronous Database Connection Helper and an Express gateway middleware. The helper ensures that if a connection is already active, the request proceeds immediately. If a connection is in progress or disconnected during a cold start, the middleware awaits the connection promise, preventing timeouts and assuring 100% request success.

3. Database Schemas & Data Integrity

Mongoose schemas govern database records, providing validation rules. Relations are maintained via references to the User model, securing customer details under individual accounts.

User Model (User Authentication)

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});
```

Customer Model (Telemetry & Indicators)

```
const CustomerSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  age: { type: Number, required: true },
  totalPurchases: { type: Number, required: true },
  lastActiveDays: { type: Number, required: true },
  satisfactionScore: { type: Number, required: true },
  churnRisk: { type: String, enum: ['High', 'Medium', 'Low'] },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
});
```

4. Key Implementation Features & Design Constraints

Development adhered to a set of specific constraints that gave the project its custom aesthetic and simplified code structure:

- **Text-Only Design System:** No emojis or graphical icons are loaded in the application. Navigation is handled via a left sidebar containing clean, bold typographic links. This creates a focused, content-first layout that looks like a clean student-level interface.
- **No Ternary Operators:** Ternary expressions (`? :`) are entirely banned in the React codebase to maintain high readability. Conditional rendering in JSX is achieved by computing helper variables or executing standard 'if/else' checks in the component body prior to the return block.
- **Deterministic Churn Scoring:** Customer risk level is calculated dynamically on the backend. High Risk is assigned if `lastActiveDays > 30 AND satisfactionScore < 5`. Low Risk is assigned if `lastActiveDays <= 14 AND satisfactionScore >= 7`. Otherwise, the customer is categorized as Medium Risk.
- **Dual-Engine AI Recommendation System:** The AI Advisor calls 'gemini-2.0-flash-lite' (the cheapest/fastest Gemini model). If the API key is throttled or lacks quota (HTTP 429), it automatically falls back to a rules-based advisor. The fallback engine evaluates customer telemetry to generate details-specific churn reasons and offers in the exact same JSON format, preventing user-facing crashes.

5. Hackathon Presentation & Slide Outline

This outline provides a structured framework to build your PPT deck and guide your live demonstration for the Epsilon TeXpedition judges.

Slide 1: Title & Pitch

RelationshipOS: Predicting Customer Churn and Automating AI Retention Actions. Hook: 'Why bleed customer revenue when you can foresee departures and act before they happen?' Introduce Epsilon TeXpedition Hackathon branding.

Slide 2: The Core Problem

Customer retention is cheaper than acquisition. Traditional businesses fail to monitor key signals: long periods of inactivity and dropping customer satisfaction. They lack tools to turn this raw data into marketing campaigns instantly.

Slide 3: Our Solution

A MERN portal that tracks customer health. Ingests raw customer activity telemetry, rates churn risk immediately, and generates custom AI-driven offers, emails, and retention actions using Gemini.

Slide 4: Technical Innovation

High-performance serverless architecture. Integrates: (1) Asynchronous connection handshakes in Vercel to bypass serverless cold starts. (2) Cost-optimized, lightweight Gemini models. (3) A transparent local fallback engine that guarantees continuous uptime during judge evaluations.

Slide 5: Design Philosophy & Rules

Designed with a content-first, high-readability approach: (1) Text-only left sidebar navigation (no icons or emojis). (2) Clean typographic layout. (3) Codebase built without ternary operators to assure readability and strict formatting.

Slide 6: Dynamic Live Demo Script

Walk through a live user story:

- Register a new account -> Log in -> Redirect to Dashboard.
- Navigate to Customers -> Create a regular client (Low Risk).
- Create an inactive client (Last Active: 45, Satisfaction: 3) -> Saves as High Risk (Red).
- Return to Dashboard -> Point out the customer is in the warning queue.
- Click 'Generate AI Actions' -> Load AI Suggestions screen -> Generate recommendations. Highlight the transition to fallback engine if quota is exceeded.

Slide 7: Technical Setup Summary

Backend: Node, Express, MongoDB Atlas, CORS, dotenv, JSONWebToken, bcryptjs. Frontend: React, Vite, Axios, Recharts (visual charts of segments). Deployed on Vercel serverless platform.

6. Deployment & Environmental Settings

To run the application locally or troubleshoot the live deployment on Vercel, ensure the following configuration files are set up correctly.

Local Setup (.env in /backend)

```
PORT=5000
MONGODB_URI=mongodb+srv://<user>:<password>@cluster0.mongodb.net/database
JWT_SECRET=hackathon_super_secret_token_12345
GEMINI_API_KEY=your_gemini_api_key
```

Local Setup (.env in /frontend)

```
VITE_API_URL=http://localhost:5000/api
```

Vercel Settings - Backend Project (relationship-os-backend)

Add the environment variables (MONGODB_URI, JWT_SECRET, GEMINI_API_KEY) in the Vercel Dashboard project settings. Rebuild the deployment after saving.

Additionally, ensure your MongoDB Atlas console has whitelisted dynamic serverless IPs by adding 0.0.0.0/0 to Network Access.

Vercel Settings - Frontend Project (relationship-os-psi)

Add the VITE_API_URL environment variable and point it to the backend deployment: <https://relationship-os-backend.vercel.app/api>. Redeploy the frontend to apply the changes.